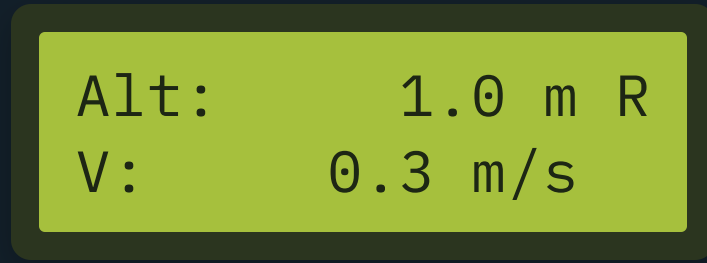
MPMC PROJECT · FINAL REVIEW

A barometric altimeter on the 8051

AT89S52, BMP280 pressure sensor and a 16×2 LCD.
Written in C, built with SDCC and flashed from Linux.



Nithish Siva Kumar · 23BCE1230 · Guide: Dr. Sivasubramanian A

OVERVIEW

What the device does

Near sea level, air pressure drops by about 12 Pa for every metre you climb. The BMP280 resolves single pascals, so the 8051 can turn pressure into height and show it on the LCD four times a second.

WORKING AT REVIEW 2

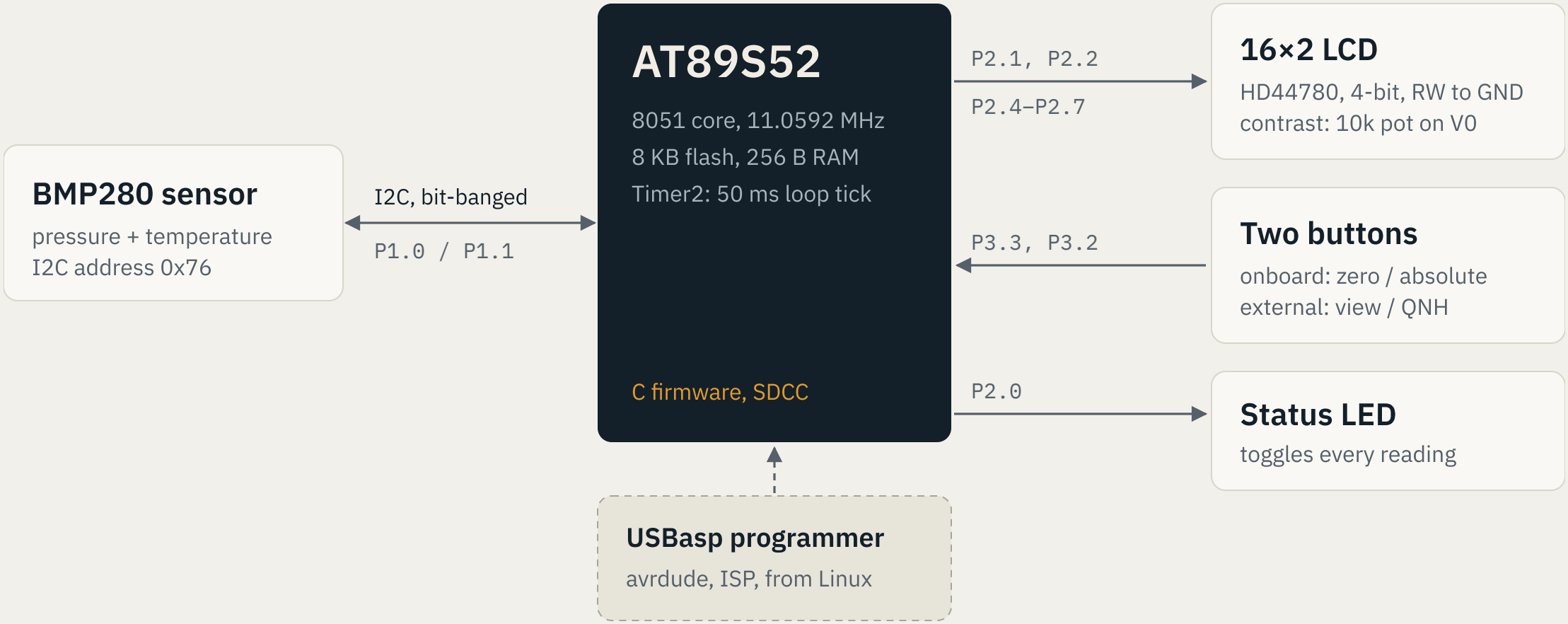
- Relative altitude: press a button to set 0 m here
- Absolute altitude against 1013.25 hPa
- Temperature and pressure on line 2
- Integer-only maths, 4 updates per second

ADDED SINCE (THE PLANNED FUTURE WORK)

- Altitude corrected for the measured air temperature
- 2-state Kalman filter, which also gives vertical speed
- Second button to enter today's sea-level pressure (QNH)
- Four display views, including a side-by-side comparison

HARDWARE

Block diagram



HARDWARE

Pin map

Signal	Pin	Notes
I2C SCL, SDA	`P1.0, P1.1`	bit-banged
LCD RS, E	`P2.1, P2.2`	control lines
LCD D4–D7	`P2.4–P2.7`	4-bit data, RW tied to GND
Status LED	`P2.0`	onboard
Zero button	`P3.3`	onboard, active low
View / QNH button	`P3.2`	external, to GND
UART TX, RX	`P3.1, P3.0`	debug build only, via MAX232

NOTES

Port 0 is unused. On this board it is open drain with no pull-ups, so it can't drive a line high.

Ports 1–3 are quasi-bidirectional. Writing 1 only enables a weak pull-up, so the buttons need no resistors and the I2C slave can always pull a line low.

LCD pins are written one bit at a time, so the LED on P2.0 is never disturbed.

FIRMWARE

How the program runs

AT POWER-UP

LCD

4-bit init sequence

I2C bus

up to 9 clocks free a stuck sensor

Sensor

probe 0x76/0x77, check ID, reset

Calibration

24 bytes, one burst read

Timer2

start the 50 ms tick

THEN, FOREVER

EVERY 50 MS TICK

Read both buttons and count how long each is held. Taps and holds trigger zero, absolute mode, the next view or the QNH editor.

EVERY 5TH TICK (250 MS)

Read raw data → Bosch compensation → Kalman step → altitude → redraw the LCD → toggle the LED.

Timer2 runs in auto-reload mode, so the period stays exactly 250 ms however long the maths takes. The Kalman model depends on a fixed time step.

THE SENSOR

Talking to the BMP280

Register	Value	Meaning
`0xD0 id`	`0x58`	BMP280 (sold as a BME280)
`0xE0 reset`	`0xB6`	soft reset, wait 5 ms
`0xF5 config`	`0x2C`	IIR filter ×8, 62.5 ms standby
`0xF4 ctrl_meas`	`0x57`	pressure ×16, temp ×2, normal mode
`0x88–0x9F`	`read`	12 factory calibration constants
`0xF7–0xFC`	`read`	raw 20-bit pressure and temperature

RAW COUNTS TO REAL UNITS

Bosch's 32-bit integer formulas combine the raw counts with the calibration constants: temperature in 0.01 °C, pressure in whole pascals (about 8 cm of height).

SDCC's `int` is 16 bits, so every term is cast to `int32_t`. Without the casts the intermediates overflow silently.

Both the sensor and SDCC store multi-byte values little-endian, so the calibration bytes go straight into a C struct.

PRESSURE TO HEIGHT · 1

The standard-atmosphere formula

```
h = 44330 × (1 − (P / P0)^0.1903)        P0 = 101325 Pa, assumes 15 °C at sea level
```

PROBLEM

The fractional power needs `powf`, which on an 8051 is a software log and exp. It pushed the program to about 7.3 of the 8 KB of flash and overflowed RAM.

LOOKUP TABLE

- A Python script evaluates the formula every 1024 Pa from 86 016 to 106 496 Pa (about +1.3 km to −430 m): 21 values in flash.
- Stored as 16-bit values with an offset. A 1024 Pa step means the 8051 interpolates with shifts, not division.
- Error against the exact formula: under 0.2 m.

The table now feeds the "ISA" comparison view. The main reading uses the temperature-corrected formula on the next slide.

Correcting for air temperature

The standard formula assumes 15 °C air. Warm air is less dense, so the same pressure drop means more height. The hypsometric equation uses the measured temperature instead:

```
h = 29.27 m/K × T × ln(P0 / P)
ln(P0/P) ≈ 2(P0 − P)/(P0 + P)  for small ratios
8051: k = 5854·(t + 27315)/8000
      h = k·(P0 − P) / ((P0 + P)/8)  [0.1 m]
```

All 32-bit integers: t in 0.01 °C, P in units of ¼ Pa. Matches the exact formula within 0.09 m (tools/check_hyps.py). This is separate from Bosch's compensation, which corrects the pressure reading for the chip's own temperature.

Air temp	Standard	Corrected
0 °C	99.5 m	94.4 m
15 °C	99.5 m	99.6 m
25 °C	99.5 m	103.0 m
32.8 °C (our room)	99.5 m	**105.7 m**
40 °C	99.5 m	108.2 m

Height for the same 11.9 hPa pressure drop. At 32.8 °C the standard formula reads 5.9 % low, so a 1 m lift would show about 0.94 m.

FILTERING · 1

A 2-state Kalman filter

State: pressure p and its rate of change v. Model: constant velocity with random acceleration, one step every 0.25 s.

```
predict   p ← p + v·dt
          P ← F·P·Fᵀ + Q
update    K ← P·Hᵀ / (H·P·Hᵀ + R)
          [p v] ← [p v] + K·(z − p)
          P ← (I − K·H)·P
```

z = measured pressure. Q: how hard the board may be moved. R: sensor noise. K is the gain that balances the two.

ON THE 8051

- The full filter runs on the chip: the covariance P is updated every step, so the gain starts high and settles as confidence grows.
- 32-bit fixed point: p, v and P scaled by 256, gains by 4096; dt = ¼ s becomes a shift.
- It filters pressure, not altitude, so re-zeroing or a new QNH never disturbs it.
- Tuning: sensor noise 0.71 Pa, acceleration 0.1 m/s². Steady-state gains 0.37 and 0.33/s.

FILTERING · 2

What the filter changes

	Old: sensor IIR ×16 + running average	New: sensor IIR ×8 + Kalman
Jitter at rest	3.8 cm	5.9 cm
Lag behind a 1 m lift	3.25 s	**1.0 s**
Vertical speed	none	**yes, ±2 cm/s at rest**

Both jitter figures are below the 0.1 m display step. The sensor's own filter was turned down so the Kalman filter, which knows about motion, does the smoothing.

HOW THESE WERE MEASURED

tools/kalman_sim.py models the sensor (noise, its IIR filter, 1 Pa rounding) and a 1 m lift. It runs a float Kalman filter next to a line-by-line copy of the C integer code: they agree within 0.3 cm.

With no serial adapter we couldn't log real data, so on the board it was checked by eye: steady at rest, follows a lift, vertical speed positive going up and negative going down.

ABSOLUTE ALTITUDE

Entering today's sea-level pressure

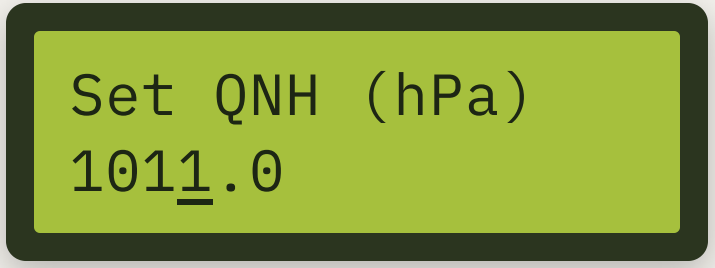
Absolute mode assumed 1013.25 hPa at sea level. Real sea-level pressure changes with the weather, and every 1 hPa of difference moves the reading by about 8 m.

QNH is the current pressure reduced to sea level. Airports publish it every half hour in their METAR reports, e.g. Q1011 = 1011 hPa. Entering it is how aircraft altimeters are set.

EDITOR, EXTERNAL BUTTON

Hold 1 s to open. Tap: current digit +1. Hold: next digit. Hold on the last digit: save and switch to absolute mode.

Values outside 870–1085 hPa (the recorded extremes) are rejected.



The value lives in RAM, so it resets to 1013.25 hPa at power-off: the AT89S52 has no EEPROM.

USING IT

Two buttons, four views

Button	Tap	Hold
Onboard, P3.3	zero here (R)	2 s: absolute (A)
External, P3.2	next view	1 s: QNH editor

Line 1 always shows the temperature-corrected altitude, with R (relative) or A (absolute) at the end. The LED toggles on every reading.





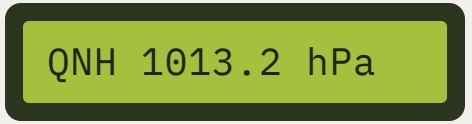
temperature, pressure



same height, 15 °C formula



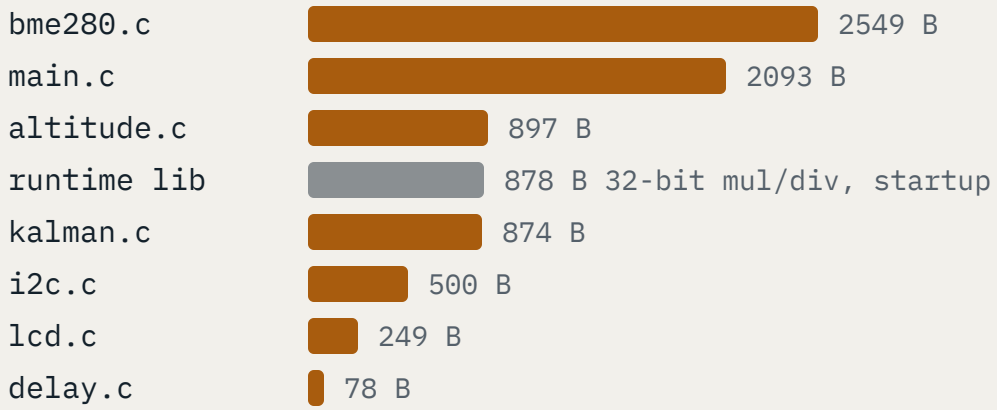
vertical speed (Kalman)



sea-level reference

RESOURCES

Fitting into 8 KB of flash and 256 B of RAM



- Calibration data and buffers in indirect RAM (`__idata`); 32-bit functions `__reentrant` so their locals use the stack.
- No `printf`, no float. UART prints only in a debug build (saves 386 B).
- With the new features we were 223 B over; a smaller 16-bit table and shared Kalman helpers made it fit.

8118 B

of 8192 B flash used

74 bytes left

63 B

of stack free

only 128 B of RAM is directly addressable

ENGINEERING

Problems and how we solved them

Problem	Fix
No I2C controller on the AT89S52	Bit-banged I2C on Port 1, with bus recovery at start-up
8-bit CPU, 16-bit `int`	Bosch's 32-bit formulas with explicit `int32_t` casts
`powf` too big	Precomputed lookup table, later the hypsometric formula in integers
Linker out of direct RAM	`__idata` for data, `__reentrant` for 32-bit functions
LCD blank, then unreadable	RW and V0 were floating. With no pot, Timer2 PWM drove V0 at 3.6 kHz; now a 10k pot.
"BME280" turned out to be a BMP280	Read the chip ID; the code supports both
Flashing seemed to hang	The programmer writes ~26 ms/byte; a full image takes 3.5 min

VERIFICATION

Testing

ON THE PC

- `check_hyps.py`: integer height formula vs exact, within 0.09 m
- `gen_alt_table.py --check`: table vs formula, within 0.17 m
- `kalman_sim.py`: C integer filter vs float, within 0.3 cm
- Read SDCC's assembly to confirm >> on a signed long keeps the sign

On the board	Result
I2C scan, chip ID	0x76, 0x58 (BMP280)
Temperature, pressure	32.8 °C, 1013.9 hPa
1 m lift after zeroing	about 1.0 m; ISA view slightly lower
Vertical speed	+ going up, – going down, ~0 at rest
QNH editor	valid value saved, 800.0 rejected
Both buttons	all taps and holds work
UART output	not tested: no RS-232 adapter

LIMITS AND NEXT STEPS

What it can't do yet

LIMITATIONS

- Absolute accuracy is about ±10 m: the sensor is ±1 hPa absolute, and QNH is rounded to 1 hPa
- Weather drift in relative mode: 0.1 hPa ≈ 0.8 m, so re-zero every few hours
- One temperature for the whole air column; fine for building heights, not for mountains
- Zero and QNH are lost at power-off
- Kalman tuning comes from simulation, not logged data
- Flash is full: 74 bytes left

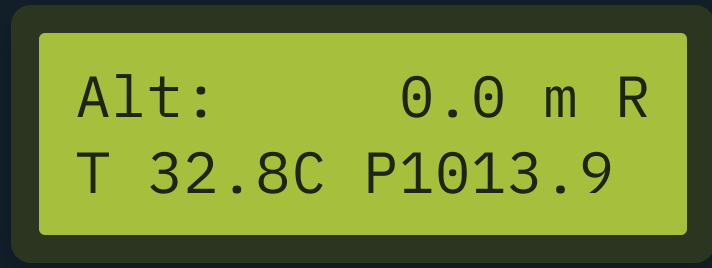
NEXT STEPS

- Swap in an AT89S8253: same 40-pin socket, 12 KB flash and 2 KB EEPROM, so settings survive power-off and there is room to grow
- Or add an AT24C02 EEPROM on the existing I2C bus
- Log raw data over a USB-serial adapter and tune the filter on real noise
- A buzzer that beeps with vertical speed, like a glider variometer
- Temperature lapse-rate correction for heights above ~1 km

DEMO

Questions?

1. Tap the onboard button: 0.0 m R
2. Lift the board 1 m: about 1.0 m
3. External button: ISA view (lower), then V while moving
4. Hold the external button, enter today's QNH: height above sea level



Nithish Siva Kumar · 23BCE1230

BACKUP

Why a phone's GPS height disagrees

In testing, a phone GPS app showed −59.7 m "MSL" while the altimeter read about 16.5 m in absolute mode. Both are "altitude", but measured from different references:

Height	Measured from	Typical error
GPS, raw	the WGS84 ellipsoid, a smooth mathematical Earth	±10–30 m vertically, worse indoors
Mean sea level	the geoid; around South India it lies roughly 60–100 m below the ellipsoid	needs a geoid model in the app
Pressure altitude (A, 1013.25)	where the pressure would be 1013.25 hPa	~8 m per hPa of weather
QNH altitude (A, QNH set)	sea level, via today's QNH	about ±10 m

The phone's −59.7 m is consistent with raw ellipsoid height that hasn't been corrected for the geoid, plus normal GPS error. For a true reference, use a surveyed elevation or a topographic map.